Timelock Shield: A Robust Defense Against MEV and Censorship Attacks in Blockchain

1st Saksham Agrawal[1,2] 2nd Vinay J. Ribeiro[1]
[1]*Department of Computer Science & Engineering, Indian Institute of Technology Bombay*, Mumbai, India
[2]*Hyperbots Inc.*, Bangalore, India

2025 Crypto Valley Conference (CVC) | 979-8-3315-8749-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CVC65719.2025.00020

***Abstract*—Blockchain's security relies on cryptographic principles to resist tampering and fraud. However, vulnerabilities such as Maximum Extractable Value (MEV) and Censorship attacks persist, where malicious actors exploit knowledge of pending transactions or bribe miners to manipulate execution. While some state-of-the-art methods aim to counter these attacks through trusted setups and fair ordering, they compromise decentralization. Alternative solutions based on commit-and-reveal are decentralized but vulnerable to DoS and deferred reveal attacks. We introduce Timelock Shield, a decentralized protocol to counteract MEV and censorship attacks. Timelock Shield utilizes timelock encryption and delayed transaction execution related to a particular smart contract. Timelock encryption ensures that the transaction contents remain confidential until a cryptographic puzzle is resolved, and delayed execution allows the state of the smart contract related to that transaction to be updated later. Crucially, the protocol operates without requiring a trusted setup, directly addressing the vulnerabilities of existing methods and significantly enhancing both security and applicability. We present a detailed game-theoretic analysis of the complex incentive structure of the protocol for various participants, demonstrating the ineffectiveness of censorship attempts and the impracticality of MEV strategies. Furthermore, we detail the implementation of Timelock Shield's key features based on Wesolowski's Verifiable Delay Function (VDF), which enables the generation of encrypted transactions, decryption, proof generation, and transaction verification. Drawing from our experimental analysis, we offer insight into the selection of VDF parameters and deadlines used in the protocol to better align with real-world computational durations.**

***Index Terms*—Blockchain, MEV Mitigation, Censorship attack, DeFi, Economics, VDF**

I. Introduction

Decentralized finance (DeFi) uses blockchain technology to create a financial system without central authorities, offering more accessibility and transparency. Blockchain acts as a secure, immutable ledger for peer-to-peer transactions, removing the need for intermediaries like banks and transforming financial services. However, a critical challenge in blockchain networks arises from the ability of miners to manipulate transaction ordering for financial gain. Miners can reorder or selectively censor transactions, exploiting Maximal Extractable Value (MEV) strategies such as front-running, back-running, and sandwich attacks.

Front-running is an MEV attack where traders exploit advanced knowledge of pending transactions to gain profit. For example, Alice places an order to buy e_1 ETH at time t_1. Front-runners scan the blockchain for such transactions and quickly submit their own, buying e_2 ETH with a higher fee at time t_2, ensuring their transaction is processed first. This increases the market price, so she pays more when Alice's order is executed. The front-runner then sells their ETH for a profit, exploiting Alice's transaction.

Back-running occurs when an attacker places a transaction immediately after a significant trade, benefiting from the resulting price movement. Sandwich attacks manipulate asset prices by strategically placing one transaction before and another after a victim's trade, forcing them to buy at a higher price and sell at a lower one. The news article [1] reveals a troubling trend: on average, $280 million is siphoned away each month through MEV attacks. This substantial financial drain highlights the issue's scale and underscores the urgency of addressing MEV practices. Such illicit activities not only compromise the system's financial well-being but also erode the trust and integrity of the consensus layer.

On the other hand, censorship attacks seriously threaten blockchain fairness, where certain participants influence miners to delay or exclude specific transactions. This manipulation disrupts the natural order of transactions, giving an unfair advantage to some while disadvantaging others. Attackers often achieve this by bribing miners or validators to prioritize their own transactions. Such actions weaken the trust and neutrality of the system, making it less reliable. Stronger security measures are needed to prevent this and ensure all transactions are treated fairly.

Existing methods, such as F3B [2] and FairBlock [3], rely on threshold encryption to address the issue of MEV. However, threshold decryption requires a certain level of trust among a "fixed number" of participants to securely manage their keys and collaborate honestly in decryption, which deviates from the idea of a "permissionless network". Furthermore, other approaches [4]- [5] depend on the fair ordering of transactions, which necessitates trust and are complex to implement in a decentralized environment. The reliance on trust in these methods poses significant challenges in achieving decentralization. Another solution is the commit-and-reveal method, which does not rely on trust. However, this approach has its problems. Adversaries may attempt to censor the revealed transaction by bribing miners to prevent its inclusion in a block, or users themselves may act maliciously by failing to

This work was supported by Hyperbots Inc. and Trust Lab (IIT Bombay). We thank Chethan Kamath (IIT Bombay) for his valuable advice and insight.

979-8-3315-8749-9/25/$31.00 ©2025 IEEE
DOI 10.1109/CVC65719.2025.00020

reveal their transactions or flood the system with a number of encrypted transactions and reveal only certain transactions that benefit them.

We present Timelock Shield, a protocol designed to mitigate MEV and censorship attacks in blockchain transactions by employing a method similar to the commit-and-reveal scheme but that avoids the issues with traditional commit-and-reveal schemes described above. Initially, a user encrypts its transaction with a key, then timelock encrypts this key, broadcasting both the encrypted transaction and the timelock encrypted key. After this transaction gets onto the blockchain, the user can reveal the key within a specified time frame; if not, a third party continuously scanning the blockchain can decrypt and broadcast the key by performing a decryption computation. Miners then update the smart contract state using the revealed key. This process ensures the transaction details remain secure until they are confirmed on the blockchain, preventing malicious actors from censorship and MEV attempts. Our solution eliminates the need for trust assumptions and can be applied to all blockchains, making it practical and usable in real-world scenarios. Unlike traditional commit-and-reveal methods, the Timelock Shield prevents censorship attacks during the reveal phase by ensuring that third parties can independently compute the necessary information even if the reveal transaction is not included in the blockchain. A detailed analysis of why third parties cannot be obstructed through bribery or other means is provided in a later section. As a result, blocking the reveal transaction becomes ineffective. Additionally, the Timelock Shield mitigates malicious user behavior, enhancing the system's overall security.

To ensure our protocol's practicality, we implemented it based on Wesolowski's Verifiable Delay Function (VDF) [6] and conducted experiments to gain insights about the protocol parameters in real-world conditions. We do not conduct a performance comparison between Timelock Shield and other existing methods, as such a comparison would be inconclusive due to fundamentally different trade-offs in security, trust, and design. Instead, our discussion of related work highlights the qualitative distinctions between our scheme and existing approaches.

Our Contributions

The main contributions of the paper are outlined below.

- We present Timelock Shield, which utilizes timelock encryption and VDFs to ensure transaction details remain confidential until confirmed on the blockchain.
- Our game-theoretic analysis of Timelock Shield's incentivization structure demonstrates that it makes MEV and censorship attacks impractical.
- We provide insights into the Timelock Shield's practicality and suggest what protocol parameter values to use in real-world scenarios through the implementation of its key features based on Wesolowski's VDF.

The following section explores MEV attacks, providing background to understand the protocol better.

II. Background

This section explores different forms of MEV and censorship attacks and how they are executed in the blockchain ecosystem. It will also examine timelock encryption and verifiable delay functions, which act as cryptographic safeguards to enhance the security and resilience of the protocol.

A. MEV Attacks

Maximal Extractable Value (MEV) is the maximum value that can be extracted from transaction ordering in a blockchain. MEV attacks exploit the ability of miners, validators, or bots to reorder, include, or exclude transactions for profit. The three common types of MEV attacks are *front-running*, *back-running*, and *sandwich attacks*.

1) Front-Running Attacks: A front-running attack is a form of unethical practice where an adversary observes a pending transaction in the mempool and uses this knowledge to submit a strategically crafted transaction with a higher gas fee, ensuring it is mined or executed before the original (victim's) transaction. The attacker benefits by anticipating and exploiting the effect the original transaction will have on the blockchain state.

2) Back-Running Attacks: Back-running is a strategy where an attacker places a transaction immediately after a known profitable transaction in order to benefit from its impact on market conditions. For example, if a large swap on a decentralized exchange causes a price shift, a back-runner can execute a transaction right after it to capture arbitrage profits.

3) Sandwich Attack: A sandwich attack is a specialized form of attack in which an attacker places one transaction before and one transaction after a target trade. This exploits the price slippage caused by the victim's trade. The attacker first front-runs the victim by buying the asset at a lower price and then back-runs by selling it at a higher price after the victim's trade moves the price.

B. Censorship Attacks

In blockchain networks, censorship attacks arise when adversaries leverage economic incentives to disrupt the fair inclusion of transactions. One of the most prevalent forms of censorship attacks is bribery attacks, particularly in Hashed Timelock Contracts (HTLCs) [7]. In such attacks, a malicious entity bribes miners to manipulate transaction ordering by prioritizing, delaying, or excluding certain transactions. For instance, consider an HTLC between two parties, A and B, where funds are locked until A provides a cryptographic proof within a specified time frame. If A fails, the funds go to B. Even if A meets the conditions, B can bribe miners to delay $A's$ transaction until the deadline passes. This prevents A from claiming the funds, allowing B to take them once the deadline expires despite A fulfilling the contract. This form of censorship attack exploits the reliance on timely transaction processing in HTLCs, emphasizing the need for stronger countermeasures to prevent bribery and ensure the integrity and fairness of blockchain transactions.

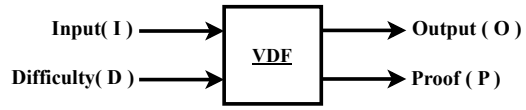

Fig. 1. VDF takes an input I and a difficulty parameter D, producing an output O and a proof P. The verification function Verify$(I, O, P) \rightarrow \{0, 1\}$ ensures that O was correctly computed within the required delay, with 1 indicating a valid proof and 0 indicating failure.

C. Verifiable Delay Functions (VDFs)

Verifiable Delay Functions, first introduced by Boneh et al. [8], are cryptographic primitives designed to enforce a computational delay while ensuring that the result remains efficiently verifiable. They require a strictly sequential number of steps to evaluate, making them resistant to parallelization, and their outputs can be publicly verified with significantly less computational effort. A VDF takes a public input I and a difficulty parameter D, determining the minimum computation time required. The function then produces an output O along with a proof P that allows efficient verification, as shown in figure 1. Formally, we express this as:

$$(O, P) = VDF(I, D) \tag{1}$$

Two popular VDF constructions are by Pietrzak [9] and Wesolowski [6], with the process comprising three key phases:

1) **Setup:** Configuring cryptographic parameters, selecting the difficulty D, and defining the underlying computational primitives.
2) **Evaluation:** Executing the function over a strictly sequential number of steps to derive O.
3) **Proof Generation:** Producing P, which enables any verifier to confirm the correctness of O efficiently.

We chose Wesolowski's construction for our protocol because its verification process only requires two exponentiations, though Pietrzak's can also be integrated with deadline adjustments.

D. Delayed Execution of Transaction (DET)

The idea of delayed execution of transactions [10] is to separate the process of adding transactions to the blockchain from executing those transactions and updating the state for the particular smart contract involved. DET is used because the decryption process in timelock encryption is computationally demanding, and miners must wait for this complex computation to complete. Instead of stalling the blockchain during this waiting period, DET allows the smart contract's state to be updated later, ensuring smoother and more efficient blockchain operations. In DET, transactions associated with a smart contract are added in sequence, but their actual validation, which involves updating the state resulting from those transactions, is delayed by τ blocks. This delay gives anyone τ blocks to calculate the necessary state for validation. For instance, if τ is set to 2, the results of transactions in the block at height k corresponding to that smart contract must be computed by the time a block is generated at height $k + 2$.

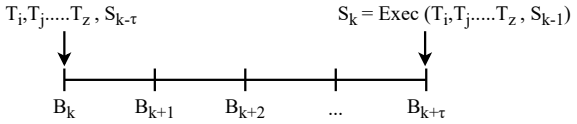

Fig. 2. Delayed Execution of Transaction. The horizontal line represents the blockchain, where transactions $T_i, T_j, \ldots, T_z$ related to a specific smart contract are included at block B_k, and the smart contract state at that point is denoted as $S_{k-\tau}$. The state of these transactions at B_k must be updated until block $B_{k+\tau}$ which is computed as $S_k = Exec(T_i, T_j, ...T_z, S_{k-1})$.

Consider blockchain as shown in figure 2. At a particular point in time, transactions $T_i, T_j, \ldots, T_z$ enter the blockchain in block B_k. These transactions are associated with a specific smart contract, and at the time of their inclusion, the contract's state is represented as $S_{k-\tau}$. However, executing these transactions does not take effect immediately upon inclusion in the block. The miners can perform computations to calculate the necessary state for validation until the block $B_{k+\tau}$. The updated state, S_k, reflects the execution results of transactions $T_i, T_j, \ldots, T_z$ on the previous state S_{k-1}. Thus, the contract's state at block $B_{k+\tau}$ is determined as:

$$S_k = \text{Exec}(T_i, T_j, \ldots, T_z, S_{k-1}) \tag{2}$$

where Exec represents the execution function that processes the transactions and updates the blockchain state accordingly. Hence, the block $B_{k+\tau}$ contains the cryptographic digest of the state resulting from the execution of all transactions up to and including those in B_k. The transactions included in blocks B_{k+1} to $B_{k+\tau-1}$ will have their effects reflected in subsequent state updates after the appropriate delay. We assume that transactions from other smart contracts do not interact with this smart contract. However, if the deadlines for all smart contracts utilizing delayed execution are synchronized, interaction becomes possible. Thus, this technique can extend to the entire blockchain, relaxing the independence assumption.

III. Related Work

MEV attacks have persisted in blockchain networks, leading to various research efforts for mitigation. Eskandari *et al.* [11] examined these attacks on decentralized applications. Dian *et al.* [12] identified the deterministic value miners can extract through transaction manipulation, while Qin *et al.* [13] quantified the profits from them.

Mitigation strategies for MEV attacks generally fall into four main categories. Firstly, there are solutions aimed at achieving fairness in transaction ordering [4]- [5]. However, many of these solutions necessitate changes to the consensus algorithm and may rely on trust assumptions. In contrast, the proposed protocol operates without altering the consensus mechanism and can be implemented as a smart contract that operates in a trustless manner. Secondly, certain approaches involve direct interaction with miners, such as the Flashbots

TABLE I
COMPARISON OF TIMELOCK SHIELD WITH EXISTING APPROACHES

Property	Commit-and-Reveal	Threshold Decryption	Timelock Shield
Trust Assumptions	Requires participants to reveal commitments honestly, introducing the possibility of strategic withholding.	Relies on a subset of key holders to correctly perform decryption without collusion or non-cooperation.	Minimizes trust assumptions, as decryption is enforced deterministically by the timelock mechanism.
Mandatory Disclosure of Reveal	Transactions can be censored if participants refuse to reveal commitments.	Colluding key holders can delay or prevent decryption, impacting protocol progress.	Timelock Shield ensures that transactions are eventually processed, mitigating censorship risks.
Autonomy	Requires active participation in commitment and reveal phases, making execution dependent on user actions.	Execution depends on the cooperation of a predefined set of key holders for decryption.	Decryption relies on a decentralized group to perform computation, ensuring progress without requiring cooperation from specific participants.
Fair Transaction Ordering	Participants can strategically time commitment reveals to influence transaction ordering.	Keyholders may delay or withhold decryption, potentially affecting the execution order.	The protocol enforces a timelock and penalizes delayed decryption, though network conditions may still influence transaction ordering.

framework [14]. In the Flashbots framework [14], transactions are aggregated by relayers and forwarded to miners for execution. However, this approach is susceptible to manipulation by relayers, who may exploit MEV opportunities themselves. Thirdly, some solutions leverage threshold decryption mechanisms involving secret management committees. Works like F3B [2], FairBlock [3], and Shutter Network [15] have sought to mitigate issues using threshold decryption. However, these approaches necessitate trust in the secret management committee. Finally, in commit-and-reveal schemes, like Namecoin [16], users send hidden transactions and then reveal them for processing. But there are issues with this method, as discussed in section I. LibSubmarine [17] takes a different approach: it requires users to create a new smart contract for every transaction they make, which isn't very efficient. Furthermore, Varun et al. introduced a machine learning-based approach to mitigate front-running, albeit with the requirement of continuous model training [13].

Several studies have also explored transaction censorship in blockchain networks. The concept of bribery attacks was first introduced by Winzer et al. [18], highlighting how malicious actors could bribe miners to gain preferential transaction ordering. To mitigate such attacks, Tsabary et al. proposed MAD-HTLC [19], which directly involves miners in the contract design. If a participant misbehaves, miners are incentivized to confiscate their funds, deterring attacks. However, Wadhwa et al. identified vulnerabilities in MAD-HTLC and introduced He-HTLC [20], a lightweight, incentive-compatible solution secure against passive and active bribery by rational miners. Our approach is different from what was discussed in their ideas. Our approach differs from these existing solutions by utilizing a combination of timelocks and verifiable delay functions (VDFs) to mitigate bribery-related threats in blockchain transaction ordering.

Timelock Shield deals with all the challenges faced by these approaches and effectively mitigates MEV and censorship attacks, as detailed in Table I. This table provides a comprehensive comparison, highlighting how our protocol overcomes the limitations of existing methods.

IV. SYSTEM OVERVIEW

This section provides an overview of the protocol, discussing its motivation, goals, and the network and threat model.

A. Goals of Protocol

In pursuit of developing a secure protocol, we now delve into the specific goals that will guide its design.

1) **Prevention of MEV & Censorship**: The protocol should include mechanisms to mitigate MEV and resist bribery.
2) **Confidentiality**: Transaction details should remain confidential until confirmed in the blockchain.
3) **Compatibility**: The protocol should seamlessly integrate with existing consensus frameworks and blockchain platforms.
4) **Decentralization Principle**: The protocol should operate without reliance on a central authority or governing body.

B. Notations

The table II represents the symbols used in the context of computation rewards, costs, transaction fees, and participant utilities. We will use these symbols in the subsequent discussions and analyses as convenient references.

C. System & Network Model

The Timelock Shield protocol involves four key participants:

- **Bribers/Front-Runners**: Potential to influence miners through financial incentives.
- **Miners**: Responsible for maintaining the blockchain.
- **Users**: Send transactions that are vulnerable to MEV or censorship. The users can choose any symmetric encryption mechanism to encrypt the transaction with a key and broadcast it to miners.
- **Third party**: These are randomly selected for each transaction using a sortition [21] algorithm. They are required to perform decryption computations. Sortition is a probabilistic algorithm that randomly selects participants

TABLE II
INTERPRETATION OF DIFFERENT SYMBOLS

This table represents the symbols used in the context of computation rewards, costs, transaction fees, and participant utilities.

Symbol	Meaning
T_1	First transaction broadcasted by user
T_2	Reveal transaction broadcasted by user
T_3	Transaction broadcasted by third party
F_1	Transaction fees for reveal transaction
F_2	Transaction fees for transaction broadcasted by third party
D	Deposit by third party
B	Fees for Bribery
I_m	Blockchain stalling penalty for miner m
U_U	Utility of User u
U_A	Utility of Attacker a
U_T	Utility of third party t
U_M	Utility of Miner m

from a larger pool, ensuring fairness and impartiality in the selection process.

Our protocol is compatible with several consensus mechanisms, including proof-of-work (as used in Bitcoin), proof-of-stake (similar to that in Ethereum), and Practical Byzantine Fault Tolerance (PBFT). Furthermore, the assumptions include that third parties are well-connected with the underlying blockchain miners and possess synchronous communication channels, ensuring timely message delivery. Specifically, if one node sends a message on the network, another node should receive it within a finite delay.

D. Threat Model

This section outlines the protocol's threat model and security assumptions. We assume that solving the timelock puzzle requires significant time for all participants. The protocol's security relies on adversaries being unable to break cryptographic schemes like VDF, VRF, and encryption mechanisms within feasible time and resources. When implementing the protocol on different blockchain systems, specific assumptions about adversary control must be considered. For PBFT blockchains, no more than c out of $3c + 1$ nodes can fail or act maliciously, ensuring consensus security. For proof-of-work blockchains, adversaries are assumed to control less than 50% of computational power. We also assume adversaries have limited computational and financial resources. All transactions related to a smart contract are encrypted, ensuring confidentiality. Blocking all encrypted transactions is considered to be infeasible, maintaining blockchain availability. The model assumes that participants behave rationally, making decisions that maximize their individual utility within the system.

V. Our Protocol

This section will discuss the details of the Timelock Shield protocol, outlining its five primary phases: *setup*, *reveal*, *computation*, *state updation*, and *reward collection*.

A. Setup Phase

During the setup phase, the user whose transaction is susceptible to attack generates a sufficiently long and random key K suitable for a standard encryption scheme. This key must have a bit length of 128 bits or more to resist brute-force attacks effectively. The transaction T_1 is then encrypted with the key K. This encryption ensures that the contents of T_1, including the input address, related smart contract, and metadata, are completely concealed. Consequently, no information can be inferred from the encrypted transaction. Following the encryption of T_1, the key K is subjected to timelock encryption as described in [22]:

1) **Composite Modulus Generation**: User begins by constructing a composite modulus
$$n \leftarrow p.q \tag{3}$$
where p and q are large, randomly selected prime numbers. Additionally, they compute Euler's totient function
$$\phi(n) \leftarrow (p-1).(q-1) \tag{4}$$
2) **Time Factor Calculation**: The user determines the time factor t by specifying the number of squares needed, which should match the desired duration for keeping the key K encrypted. For blockchains vulnerable to forking, t should align with the blockchain's confirmation time. In Section VII, we analyze computation times to provide insights on choosing an appropriate t based on the desired encryption duration.
3) **Random Base Selection**: The user selects a random base value, denoted by a, from the set of residues modulo n such that $1 < a < n$.
4) **Key Encryption**: After selecting a, they use a and t to encrypt the key K, resulting in
$$C_K \leftarrow K + a^{2^t} \pmod{n} \tag{5}$$
Here, any operation can be performed with the key, but an addition operation is used for simplicity. To set the trapdoor, they first compute
$$b \leftarrow 2^t \pmod{\phi(n)} \tag{6}$$
Then they compute
$$x \leftarrow a^b \pmod{n} \tag{7}$$
5) **Timelock Puzzle Presentation**: The user then presents the encrypted transaction with the timelock puzzle (n, a, t, C_K, C_{T_1}) as the final output, where C_{T_1} is the encrypted data of the transaction. They also generate $l = H_{\text{prime}}(x+a)$, where $H_{\text{prime}}(m)$ denotes the smallest prime number greater than or equal to $H(m)$, and include it in this transaction, ensuring the removal of any intermediate variables such as p and q to maintain security. The whole process is executed off-chain, eliminating the need for on-chain computation and reducing blockchain overhead. This transaction is broadcasted to miners with competitive transaction fees to ensure it is

included as soon as possible. The use of l will be detailed in subsequent phases.

The user must also lock the computation reward R in the transaction, which prevents malicious users from flooding the blockchain with junk transactions, overcoming the limitation of naive commit-and-reveal schemes. This transaction must be added to the blockchain before a deadline D_1, which is set shorter than the decryption time by the fastest computer to prevent off-chain decryption (see Figure 3). More details about D_1 are provided in Section VII. The time factor t should be constrained within a predefined range to enable miners to verify that decryption is computationally feasible within a specific time window. By establishing an expected interval $[t_{\min}, t_{\max}]$, the protocol ensures that valid transactions can be decrypted within this timeframe, thereby mitigating the risk of unverifiable or non-decryptable submissions. If a user fails to adhere to this requirement, the computation reward R will be forfeited as a penalty. Additionally, to ensure anonymity and prevent detection or front-running, the user should broadcast the transaction in a manner that dissociates it from their identity and obfuscates its origin.

B. Reveal Phase

After broadcasting the encrypted transaction, the user must reveal the key K to miners before the deadline D_2 to decrypt T_1. In traditional commit-and-reveal schemes, users may skip the reveal if unprofitable, but in our protocol, failing to reveal or reveal the wrong key incurs a penalty of computation reward R. The user reveals K by submitting T_2, which should be done after T_1 is confirmed in the blockchain. The deadline D_2 depends on the blockchain's vulnerability to forks. Most users will reveal the key, skipping the computation phase. The computation reward R is unlocked if the key is revealed. Failure to reveal K by D_2 moves the protocol to the third phase, and any reveal transaction broadcasted after D_2 will be invalid. Moreover, even if an attacker attempts to block the reveal transaction, they will not achieve their objective, as the protocol will transition to the next phase regardless of the delay. Incentives for miners and users to include and reveal the transaction are discussed in Section VI.

C. Computation Phase

If the user does not reveal the key K in the reveal phase before D_2, the computation phase of the protocol begins (refer to Figure 3). In this phase, third parties scan the blockchain. They decrypt the transaction through computation if they observe that T_2 is not present on the blockchain. The required computation is performed off-chain, minimizing on-chain resource utilization. Interested parties lock a deposit D during the reveal phase as a commitment to solving the puzzle and decrypting the transaction for the computation reward R. A sortition function selects participants from, say, n users. This function uses the Verifiable Random Function (VRF) [21] to probabilistically and securely select a small committee, ensuring fairness and verifiability by proving whether a participant is part of the committee. The identities of the selected users remain confidential to prevent external interference. Once the committee is chosen, they performs repeated squaring t times of a to uncover the original message:

$$x \leftarrow a^{2^t} \pmod{n} \tag{8}$$

After completing the computation, they generate a proof using the concept of Wesolowski's VDF as follows. The third party reads the value of l from the blockchain and computes:

$$\pi \leftarrow a^{\lfloor 2^t / l \rfloor} \pmod{n} \tag{9}$$

The time required for proof generation depends on the number of processing cores used; more cores reduce the computation time. The third parties generate a transaction T_3, broadcasting the result and proof to all miners before the deadline D_3. To prevent bribery, they cannot reveal the VRF until the state update phase of the protocol. This ensures that the briber cannot know the selected committee members beforehand. After the state update phase, the VRF can be broadcast anytime, and the deposit D will be unlocked, provided the participant has behaved honestly. The deadline D_3 should be kept such that even the slowest computer can do the computation of decryption. Since the committee size is small, the reward is distributed equally among all committee members who submit the correct proof to the miners. Section VI discusses third parties' actions and identifies their dominant strategy.

D. State Updation Phase

At the deadline D_3, miners must perform one of the two actions while generating the block (shown in Figure 3). If the user reveals the key K in transaction T_2 during the reveal phase, miners update the state when generating the next block using T_2. If the user does not reveal the key, miners must read the proof and key K provided by a third party in transaction T_3. When miners read the message from T_3, they must verify the decrypted message y' broadcasted by the third party using the previously published value of l. The verification process can be implemented using a smart contract, ensuring automated and trustless validation. Verification is done using the following algorithm proposed in Wesolowski's VDF:

$$y \leftarrow \pi^l \cdot a^r \pmod{n} \tag{10}$$

where $r = 2^t \pmod{l}$. Miners check if $y' == y$.

At D_3, miners must execute the smart contract to update the state, as failing to do so would prevent them from generating the next block. Blockchain progression halts if no miner performs this update, leading to an economic penalty of I_m. This ensures that the protocol progresses as intended and maintains the integrity and security of the blockchain. Moreover, the verification process incurs minimal on-chain computational overhead, as Wesolowski's proof verification is highly efficient.

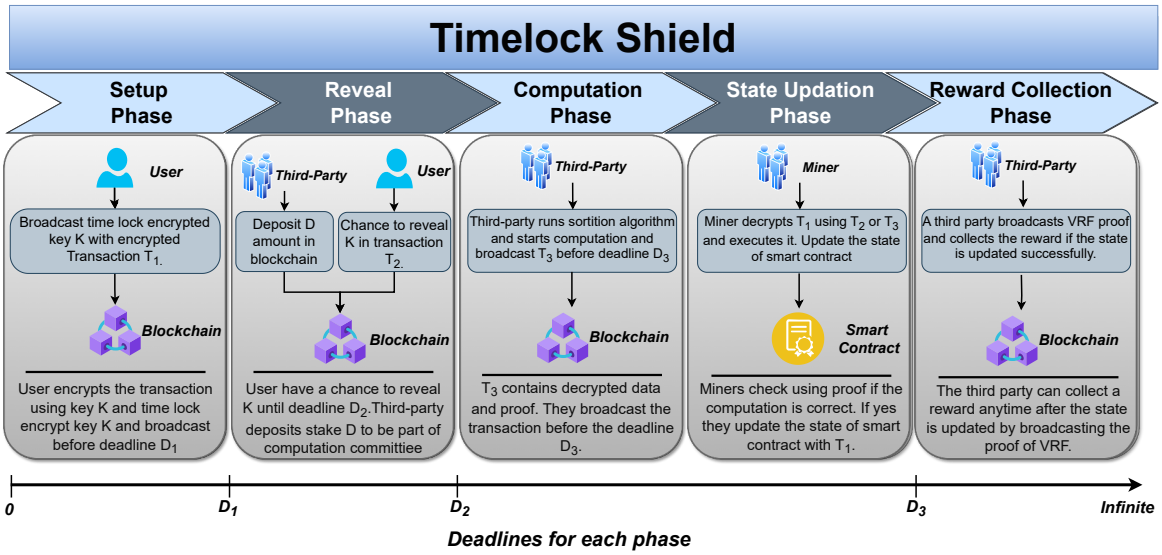

Fig. 3. The timeline of the Timelock Shield protocol, showing the required actions of each participant at every phase and indicating that each phase must be completed before the corresponding deadline.

E. Reward Collection Phase

The third party is not allowed to reveal the VRF till D_3. They can collect their reward once the state updation phase is completed after D_3. The reward will be distributed among the parties who broadcast correct proof before the deadline of D_3. If they reveal the VRF to the briber, they could accept bribes for not performing the computation. Hence, violating this rule results in losing the locked stake D and being banned from future computations.

F. Security Measures

In this section, we will see some of the critical inequalities that need to be maintained for our protocol to work efficiently without any problems and prevent MEV and censorship attacks in the blockchain. Suppose a third party incurs cost C for the puzzle computation, and the user provides a computation reward R. Here, we consider that the third party that deposits D is eligible to be a part of the solving committee. The transaction fees of the reveal transaction T_2 is F_1, and the fees for the third party's transaction T_3 is F_2. We establish the following conditions:

$$R > F_1 \tag{11}$$

This condition ensures that users prefer paying fees for the reveal transaction over providing the computation reward due to its higher value.

$$R - n * C > F_2 \tag{12}$$

This condition ensures that the third party gets rewarded for performing the computation, where n denotes the number of participants in the third party who broadcast the correct proof, which is expected to be very small.

$$B < I_m \tag{13}$$

This condition guarantees that the amount for bribery remains lower than the penalty in the event of a blockchain stall to the miners.

$$D > B \tag{14}$$

This condition indicates that if a third party shows malicious behavior, they are penalized more than rewarded.

By encrypting transactions, the content remains inaccessible to potential attackers. These transactions are revealed after being confirmed in the blockchain, ensuring security against forking attacks. While analyzing participant incentives, we will demonstrate that MEV and censorship attacks are not feasible in our protocol.

VI. Incentive Analysis

We model the interactions between participants in our system as a strategic game where each player selects an action to maximize their utility. The system consists of four types of participants: users, third parties, miners, and attackers. We analyze their decision-making using dominant strategies, incentive compatibility, and rational deviations.

A. Game Definition

The game is represented as a tuple:

$$G = \langle \mathcal{P}, \mathcal{S}, \mathcal{U} \rangle$$

where:

- $\mathcal{P}$ is the set of players: {users, third parties, miners, attackers}.

- $\mathcal{S}_i$ is the set of strategies available to each player $i \in \mathcal{P}$.
- $\mathcal{U}_i : \mathcal{S} \to \mathbb{R}$ is the utility function mapping strategy profiles to real-valued payoffs for each player.

Each player selects a strategy $s_i \in \mathcal{S}_i$ to maximize their respective utility function U_i. We analyze the best response strategies for each participant.

B. Users' Strategy

Every transaction related to a specific smart contract will be encrypted, requiring users to encrypt their initial transaction T_1. This encryption prevents attackers from deciphering the transaction content. After encryption, users can either reveal the key K or keep it concealed until the deadline D_2.

Players: $U \in \mathcal{P}$ (Users)
Strategy Set:

$$\mathcal{S}_U = \{\text{Reveal key } K,\ \text{Do not reveal key } K\}$$

Utility Function:

- If the user reveals the transaction key K, they incur a transaction fee F_1:

$$U_U(\text{Reveal}) = -F_1 \tag{15}$$

- If the user does not reveal the key, a third party will decrypt it at cost R, which is deducted from the user's initial commit:

$$U_U(\text{Not Reveal}) = -R \tag{16}$$

Since $R > F_1$, rational users will always prefer to reveal K themselves rather than incur the higher cost associated with third-party decryption. Hence, the user's **dominant strategy** is to reveal key K.

C. Third Parties' Strategy

Third parties are responsible for decryption computations if the user does not reveal the T_2 transaction. Hence, third parties have two choices: perform the calculation and reveal the key K or accept a bribe and refrain from performing the computation.

Players: $T \in \mathcal{P}$ (Third Parties)
Strategy Set:

$$\mathcal{S}_T = \{\text{Perform decryption}, \text{Do not perform decryption}\}$$

Utility Function:

- If the third party performs decryption, they receive a reward R but incur computation cost C and transaction fee F_2:

$$U_T(\text{Perform Decryption}) = R - C - F_2 \tag{17}$$

- If they do not compute and accept a bribe B, they lose their stake deposit D:

$$U_T(\text{Not Perform Decryption}) = B - D \tag{18}$$

The penalty D is sufficiently large such that $B - D < 0$. Hence, the third party's **dominant strategy** is to perform decryption.

D. Attackers' Strategy

An adversary may attempt to disrupt the protocol by bribing either miners or third parties.

Players: $A \in \mathcal{P}$ (Attackers)
Strategy Set:

$$\mathcal{S}_A = \{\text{Bribe miners}, \text{Bribe third parties}, \text{Do nothing}\}$$

Utility Function:

- If the attacker bribes miners, they incur a cost B, but the attack is ineffective since third parties will still decrypt and miners must update the smart contract state at deadline D_3:

$$U_A(\text{Bribe Miners}) = -B \tag{19}$$

- If the attacker bribes a third party, they cannot determine the composition of the computation committee. If the attacker bribes all third parties, the miners must update the smart contract state, rendering the bribe ineffective:

$$U_A(\text{Bribe Third Parties}) = -B \tag{20}$$

- If the attacker does nothing, their utility remains zero:

$$U_A(\text{Do Nothing}) = 0 \tag{21}$$

Since bribing always results in a strictly negative payoff, the attacker's **dominant strategy** is to do nothing.

E. Miners' Strategy

Miners face two strategic choices: update the smart contract's state using either T_1 or T_2 or refrain from updating it.

Players: $M \in \mathcal{P}$ (Miners)
Strategy Set:

$$\mathcal{S}_M = \{\text{Update blockchain}, \text{Do not update blockchain}\}$$

Utility Function:

- If miners update the blockchain state, they receive transaction fees F_1 or F_2:

$$U_M(\text{Update}) = +F_1 \text{ or } +F_2 \tag{22}$$

- If they do not update the blockchain, they gain no transaction fees and potentially lose mining incentives. Consequently, another miner updates the state and generates the block or the blockchain stalls.:

$$U_M(\text{Not Update}) = -I_M \tag{23}$$

Since mining incentives are always positive, the miner's **dominant strategy** is to update the blockchain state.

F. System Stability and Incentive Compatibility

Given the dominant strategies of all players, the system is incentive-compatible, meaning that no rational participant has an incentive to deviate unilaterally.

- **Users reveal transactions** ($S_U = \text{Reveal}$)
- **Third parties compute decryption** ($S_T = \text{Compute}$)
- **Miners update the blockchain** ($S_M = \text{Update}$)
- **Attackers do nothing** ($S_A = \text{Do Nothing}$)

Thus, the **strategic outcome** of the game is:

$$(S_U^*, S_T^*, S_M^*, S_A^*) = (\text{Reveal}, \text{Compute}, \text{Update}, \text{Do Nothing})$$

Theorem 1 (System Stability): Under the given incentive structure, the system remains stable because:

1) Users strictly prefer to reveal transactions, minimizing their cost.
2) Third parties are incentivized to compute and reveal the key, as any deviation is penalized.
3) Miners always prefer updating the blockchain, as it maximizes their profit.
4) Attackers have no rational incentive to attempt bribery.

Proof. Since all players follow their **dominant or incentive-compatible strategies**, any deviation leads to a lower payoff. This ensures that rational participants do not deviate, maintaining the security and fairness of the system. □

G. Effectiveness of Protocol in Achieving Objectives

This section thoroughly evaluates how effectively Timelock Shield achieves its objectives. It is designed to tackle key challenges, including MEV, bribery, denial-of-service (DoS) attacks, censorship, and trust assumptions, while ensuring security, fairness, and efficiency.

A primary goal of the protocol is to safeguard transactions against MEV and censorship. To achieve this, transactions are encrypted before submission, ensuring their confidentiality and preventing unauthorized manipulation before confirmation. Since the transaction content remains hidden until decryption, no external party can exploit its information for unfair advantages. Additionally, the protocol includes punitive measures for users who broadcast inaccurate transaction details, discouraging misinformation and ensuring data integrity. Censorship attempts are also deterred through a mechanism that imposes negative rewards on malicious actors, making such attacks unprofitable and reducing the risk of manipulation.

Timelock Shield is designed to be fully decentralized, eliminating the need for trust in any single party and avoiding any single points of failure. Unlike commit-and-reveal, which relies on user cooperation, this protocol enforces execution autonomously through a timelock mechanism. This prevents adversarial behaviors such as selective withholding of decryption keys, front-running, or collusion.

Confidentiality is another fundamental aspect of our design. We employ encryption techniques to protect transactions, ensuring they remain secure until confirmation. The use of timelock encryption guarantees that decryption occurs only after the transaction is recorded on the blockchain, preventing premature disclosure.

Furthermore, the protocol is designed for broad compatibility with various consensus algorithms, allowing seamless integration across blockchain systems. Timelock Shield fulfills its objectives by mitigating key challenges such as MEV, bribery, censorship, and DoS attacks while ensuring trustlessness, autonomy, and confidentiality.

VII. Practical Aspects

We implemented Timelock Shield using Wesolowski's VDF construction, allowing the user to encrypt, third parties to decrypt, and the miners to verify the correctness of the third party's computations efficiently. The algorithms were developed in C++ utilizing the GNU Multiple Precision Arithmetic Library (GMP).

The timelock encryption operation is performed off-chain using the algorithm 1. The encrypted transaction is broadcasted to miners for inclusion in the blockchain. Miners must include this transaction before the deadline D_1, which must be set to be shorter than the time required by the fastest computer to solve the puzzle.

Algorithm 1 Time-Lock Encryption using Rivest's Method

procedure ENCRYPTION(K, t)
  $n \leftarrow p \cdot q$ where p and q are large primes
  $\phi(n) \leftarrow (p-1)(q-1)$
  Select random a such that $1 < a < n$
  $b \leftarrow 2^t \bmod \phi(n)$
  $x \leftarrow a^b \bmod n$
  $C_K \leftarrow K + x \bmod n$
  $l \leftarrow H_{\text{prime}}(x + a)$
  return C_K, n, a, l, t
end procedure

Algorithm 2 Transaction Decryption and Proof Generation

procedure TRANSACTIONDECRYPTIONPROOF(n, a, t, l)
  Scans the blockchain for T_2.
  if T_2 is not on the blockchain **then**
    if party is part of computation committee **then**
      $y \leftarrow a$
      $tt \leftarrow t$
      while $tt \geq 0$ **do**
        $y \leftarrow y^2 \ mod \ n.$
        $tt - -$
      end while
      read l from blockchain
      $\pi \leftarrow a^{\lfloor 2^t / l \rfloor} \ mod \ n$
    end if
  end if
  return π, y
end procedure

A third party manages decryption and proof generation off-chain using algorithm 2, with the results also broadcasted as transactions. This transaction should be broadcasted before the

deadline D_3. The deadline D_3 should be such that even the slower CPU can decrypt the transaction before this deadline. Once these transactions are broadcasted, the smart contract verifies which transactions are on the blockchain, confirms the computations performed by the third party using the algorithm 3, and updates the state accordingly.

Algorithm 3 Verification using Smart Contract

procedure VERIFICATION(n, y, a, t, π, l, C_K)
  Scans the blockchain for T_2
  if T_2 is on the blockchain **then**
    Read key K from T_2
    Update the state of related smart contract
  else
    $r \leftarrow 2^t \ mod \ l$
    if $y = \pi^l a^r mod \ n$ **then**
      Read key K from T_3
      Update the state of related smart contract
    end if
  end if
end procedure

After implementing all algorithms, mapping VDF parameters and deadlines to real-world durations is the next challenge. We conducted experiments to analyze the deadlines D_1, D_2, D_3, and the parameter t. The deadline D_2 should be based on the confirmation time of the underlying blockchain.

To determine appropriate deadlines D_1 and D_3 and t, we carried out experiments on five machines with the following specifications:

- **Machine 1 (M1):** Intel Iris Plus, 1100 MHz, 8 GB RAM, macOS Sonoma 14.5.
- **Machine 2 (M2):** AMD Ryzen 5 5500U, 4056 MHz, 8 GB RAM, Ubuntu 22.04.
- **Machine 3 (M3):** Intel Core i5, 3900 MHz, 24 GB RAM, Ubuntu 23.04.
- **Machine 4 (M4):** 12th Gen Intel Core i5, 4400 MHz, 32 GB RAM, Ubuntu 23.04.
- **Machine 5 (M5):** AMD Ryzen 7 5700G, 4673 MHz, 16 GB RAM, Ubuntu 22.04.

	Number of Squarings Needed (t)				
Time	**M1**	**M2**	**M3**	**M4**	**M5**
3 seconds	1.249	1.604	1.662	2.646	2.700
12 seconds	5.293	6.360	6.552	10.60	10.70
1 minute	28.24	31.85	33.18	53.12	53.10
3 minutes	95.50	96.91	99.07	158.8	162.3
20 minutes	688.9	640.4	668.4	1090	1077

TABLE III
VALUES OF t (IN MILLIONS) CORRESPONDING TO DIFFERENT TIMES, CALCULATED BY MULTIPLYING THE NUMBER OF SQUARES PER SECOND BY THE TOTAL TIME.

We determined different values of t for various confirmation times of different blockchains required by each machine. This was done by calculating the number of squares per second each machine can perform and multiplying it by the confirmation time. The resulting values of t for different machines are shown in Table III. Next, we selected the lowest value of t, corresponding to the slowest CPU, and ran our implementation of the VDF for that parameter value. Observations were taken for each value of t five times on each machine, and the average time was considered due to the low standard deviation. The observations are shown in Table IV.

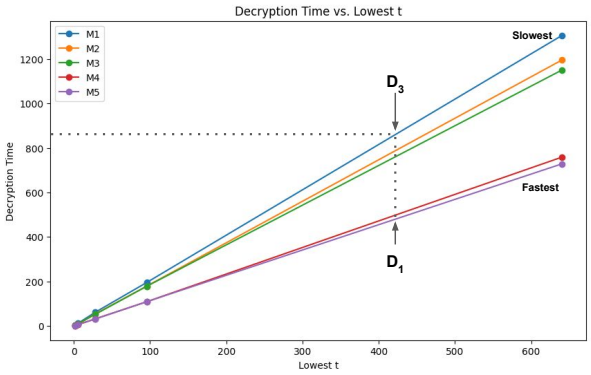

Fig. 4. Graph of Decryption Time vs. Lowest t. The user will choose value t and deadline D_1 so that the slowest computer can decrypt the message before D_3.

	Time Taken (in seconds)				
Lowest t	**M1**	**M2**	**M3**	**M4**	**M5**
1.249	3.001	2.421	2.330	1.432	1.431
5.293	11.82	6.826	9.880	6.058	6.058
28.24	62.01	54.63	52.71	32.30	31.467
95.50	195.3	179.1	178.2	108.9	108.9
640.4	1305.7	1195.42	1150.37	758.9	728.7

TABLE IV
ACTUAL DECRYPTION TIMES (IN SECONDS) CORRESPONDING TO THE LOWEST t (IN MILLIONS) FOR EACH MACHINE.

The graph in Figure 4 was created using the values from Table IV. The user should set the value of t so that the slowest third parties can decrypt the message before D_3. Additionally, the deadline D_1 should be set such that even the fastest computer cannot solve the puzzle before this deadline for that value of t, as indicated in Figure 4. This mapping for deadlines is recommended to enhance security against attacks and prevent early decryption.

VIII. CONCLUSION

In conclusion, the Timelock Shield protocol provides an effective solution for mitigating MEV and censorship attacks in blockchain systems. VDFs and delayed transaction execution (DET) keep transaction details hidden until confirmed, preventing exploitation. Furthermore, the design does not rely on trust in individual participants, adhering to the principles of decentralization and addressing the limitations and vulnerabilities of existing methods. The analysis of the incentive structure confirms that MEV and bribery become economically unfeasible, thereby enhancing the protocol's security.

The implementation shows the protocol is practical and efficient, as encryption and decryption processes are performed off-chain and align with the performance requirements of real-world blockchain systems.

REFERENCES

[1] Edvardas Mikalauskas. $280 million stolen per month from crypto transactions - cybernews.

[2] Haoqian Zhang, Louis-Henri Merino, Ziyan Qu, Mahsa Bastankhah, Vero Estrada-Galinanes, and Bryan Ford. F3b: A low-overhead blockchain architecture with per-transaction front-running protection, 2023.

[3] Peyman Momeni, Sergey Gorbunov, and Bohan Zhang. Fairblock: Preventing blockchain front-running with minimal overheads. Cryptology ePrint Archive, Paper 2022/1066, 2022. https://eprint.iacr.org/2022/1066.

[4] Mahimna Kelkar, Fan Zhang, Steven Goldfeder, and Ari Juels. Order-fairness for byzantine consensus. Cryptology ePrint Archive, Paper 2020/269, 2020. https://eprint.iacr.org/2020/269.

[5] Klaus Kursawe. Wendy grows up: More order fairness. In *Financial Cryptography and Data Security. FC 2021 International Workshops: CoDecFin, DeFi, VOTING, and WTSC, Virtual Event, March 5, 2021, Revised Selected Papers*, page 191–196, Berlin, Heidelberg, 2021. Springer-Verlag.

[6] Benjamin Wesolowski. Efficient verifiable delay functions. Cryptology ePrint Archive, Paper 2018/623, 2018. https://eprint.iacr.org/2018/623.

[7] Poon and Dryja. The bitcoin lightning network: Scalable off-chain instant payments. 2016.

[8] Dan Boneh, Joseph Bonneau, Benedikt Bünz, and Ben Fisch. Verifiable delay functions. Cryptology ePrint Archive, Paper 2018/601, 2018. https://eprint.iacr.org/2018/601.

[9] Krzysztof Pietrzak. Simple verifiable delay functions. Cryptology ePrint Archive, Paper 2018/627, 2018. https://eprint.iacr.org/2018/627.

[10] Sourav Das, Nitin Awathare, Ling Ren, Vinay J. Ribeiro, and Umesh Bellur. Tuxedo: Maximizing smart contract computation in pow blockchains. *Proc. ACM Meas. Anal. Comput. Syst.*, 5(3), December 2021.

[11] Shayan Eskandari, Mahsa Moosavi, and Jeremy Clark. Sok: Transparent dishonesty: Front-running attacks on blockchain. pages 170–189, 02 2019.

[12] Philip Daian, Steven Goldfeder, Tyler Kell, Yunqi Li, Xueyuan Zhao, Iddo Bentov, Lorenz Breidenbach, and Ari Juels. Flash boys 2.0: Frontrunning in decentralized exchanges, miner extractable value, and consensus instability. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 910–927, 2020.

[13] Kaihua Qin, Liyi Zhou, and Arthur Gervais. Quantifying blockchain extractable value: How dark is the forest? *CoRR*, abs/2101.05511, 2021.

[14] Flashbots. Flashbots — flashbots.net. https://www.flashbots.net/. [Accessed 11-12-2023].

[15] Shutter Network. Introducing shutter network - combating front running and malicious mev using threshold cryptography, Feb 2022.

[16] Harry A. Kalodner, Miles Carlsten, Paul Ellenbogen, Joseph Bonneau, and Arvind Narayanan. An empirical study of namecoin and lessons for decentralized namespace design. In *14th Annual Workshop on the Economics of Information Security, WEIS 2015, Delft, The Netherlands, 22-23 June, 2015*, 2015.

[17] Lorenz Breidenbach, Philip Daian, Florian Tramèr, and Ari Juels. Enter the hydra: Towards principled bug bounties and exploit-resistant smart contracts. Cryptology ePrint Archive, Paper 2017/1090, 2017. https://eprint.iacr.org/2017/1090.

[18] Fredrik Winzer, Benjamin Herd, and Sebastian Faust. Temporary censorship attacks in the presence of rational miners. *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*, pages 357–366, 2019.

[19] Itay Tsabary, Matan Yechieli, Alex Manuskin, and Ittay Eyal. Mad-htlc: Because htlc is crazy-cheap to attack, 2021.

[20] Sarisht Wadhwa, Jannis Stoeter, Fan Zhang, and Kartik Nayak. He-htlc: Revisiting incentives in htlc. Cryptology ePrint Archive, Paper 2022/546, 2022. https://eprint.iacr.org/2022/546.

[21] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *Proceedings of the 26th Symposium on Operating Systems Principles*, SOSP '17, page 51–68, New York, NY, USA, 2017. Association for Computing Machinery.

[22] Ronald L. Rivest, Adi Shamir, and David A. Wagner. Time-lock puzzles and timed-release crypto. Technical Report Technical memo MIT/LCS/TR-684, MIT Laboratory for Computer Science, 1996. (Revision 3/10/96).